Gehrke Thesis Chapter 3.2: Cumulant Collision Operator Implementation

Equation-by-Equation Mapping to CUDA Code
(Eq. 3.59 – 3.121)

Updated: General ω_6, ω_7, ω_8 — Thesis Original Form

Implementation Report for GILBM–Cumulant Project

March 13, 2026

Contents

1 Overview **3**

2 Stage 0: Well-Conditioning (§3.2.1) **3**
2.1 Eq. 3.59 — Well-Conditioned PDFs . . . 3
2.2 Eq. 3.60–3.63 — Density and Velocity . . . 4

3 Stage 1: Forward Chimera Transform (Eq. 3.64) **5**

4 Stage 2: Central Moments → Cumulants **5**
4.1 4th-Order Off-Diagonal (Eq. J.16) . . . 6
4.2 4th-Order Diagonal (Eq. J.17) and Higher Orders . . . 6

5 Stage 3: Relaxation (Collision in Cumulant Space) **6**
5.1 Eq. 3.65–3.66 — Relaxation Rates . . . 6
5.2 Relaxation Rate Definitions (ω_2, ω_6–ω_{10}) . . . 7
5.3 Eq. 3.67–3.68 — Conservation (0th, 1st Order) . . . 8
5.4 Eq. 3.69 — Off-Diagonal 2nd Order . . . 8
5.5 Eq. 3.70–3.75 — Diagonal 2nd Order with Galilean Correction . . . 8
5.6 Eq. 3.76–3.82 — Third-Order Relaxation . . . 10
5.7 Eq. 3.83–3.85 — Fourth-Order Diagonal (Thesis Original Form) . . . 11
5.8 Eq. 3.86–3.88 — Fourth-Order Off-Diagonal (Thesis Original Form) . . . 12
5.9 AO Mode: Fourth-Order Relaxation . . . 13
5.10 Eq. 3.90–3.91 — Fifth and Sixth Order . . . 14

6 Employed Cumulant Variants (§3.2.3) **14**
6.1 Eq. 3.92–3.102 — All-One (C-AO) Variant . . . 14
6.2 Eq. 3.103–3.107 — Parameterized (C-P/WP) Variant . . . 15
6.3 Eq. 3.108–3.110 — High-Re Limit Conservation . . . 16
6.4 Eq. 3.111–3.115 — Lambda Limiter (Regularization) . . . 16
6.5 Eq. 3.116–3.121 — Special Case $\omega_6 = \omega_7 = \omega_8 = 1$ . . . 17

7 Stage 4: Cumulants → Central Moments (Inverse) **18**
7.1 4th-Order Off-Diagonal Inverse (Eq. J.16 inverse) . . . 18
7.2 4th-Order Diagonal, 5th-Order, 6th-Order Inverse . . . 18

8 Stage 5: Backward Chimera and Body Force **18**
8.1 Strang-Splitting Body Force (Thesis §3.2.1, p. 48) 18
8.2 Backward Chimera ($x \to y \to z$) 19

9 Summary Table: Equation → Code Mapping **19**

10 Errata and Bug Fixes **20**

11 Appendix: Index Alias Table **21**

1 Overview

This document provides a detailed, equation-by-equation mapping between **Gehrke's PhD thesis** (Chapter 3.2, "Cumulant Collision Operator Implementation") and the CUDA implementation in `cumulant_collision.h`.

Source files:

- `Cumulants/cumulant_collision.h` — GPU collision kernel (∼813 lines)
- `Cumulants/cumulant_constants.h` — Chimera indices & K constants (201 lines)

Collision pipeline:

Stage 0 Macroscopic quantities + Well-conditioning (Eq. 3.59–3.63)

Stage 1 Forward Chimera: $f_\zeta \to \kappa_{\alpha\beta\gamma}$ (Eq. 3.64)

Stage 2 Central moments → Cumulants (Eq. J.16–J.19)

Stage 3 Relaxation in cumulant space (Eq. 3.67–3.91)

Stage 4 Cumulants → Central moments (Eq. J.16–J.19 inverse)

Stage 5 Backward Chimera: $\kappa_{\alpha\beta\gamma} \to f^*_\zeta$

Key design choices in this implementation:

1. All thesis formulas are implemented in their **original, unsimplified form**.
2. ω_6, ω_7, ω_8 are **retained as independent parameters** (not preset to 1).
3. Override via compile-time macros: `CUM_OMEGA6`, `CUM_OMEGA7`, `CUM_OMEGA8`.
4. Default values $\omega_6 = \omega_7 = \omega_8 = 1$ recover the standard WP/AO behavior.
5. Thesis Eq. 3.89 contains a confirmed **typographic error** ($C_{110} \leftrightarrow C_{011}$ swapped); the code uses the corrected Chapman–Enskog mapping.

2 Stage 0: Well-Conditioning (§3.2.1)

2.1 Eq. 3.59 — Well-Conditioned PDFs

Thesis Eq. 3.59

To reduce rounding errors, the well-conditioned PDFs are defined by subtracting the weights:

$$f_\zeta = f^c_\zeta - w_{ijk}\,, \tag{3.59}$$

where f^c_ζ is the "compressible" PDF and w_{ijk} are the D3Q27 weights (Eq. 3.15). This is based on $\rho = \rho_0 + \delta\rho$ with $\rho_0 = 1$.

Implementation: L240–244

```
// 0d. Well-conditioning: f = f - w
double m[27];
for (int i = 0; i < 27; i++) {
    m[i] = f_in[i] - GILBM_W[i];
}
```

Note

The variable `m[27]` stores the well-conditioned values throughout the entire collision. The auxiliary variable `drho = rho - 1.0` (L247) is also derived from this formulation. After collision (L710–712), the weights are restored: `f_out[i] = m[i] + GILBM_W[i]`.

2.2 Eq. 3.60–3.63 — Density and Velocity

Thesis Eq. 3.60–3.63

The density fluctuation $\delta\rho$ is computed by pairwise summation of well-conditioned PDFs grouped into vertices, medians, principal axes, and center (Eq. 3.60). The velocity components:

$$u = \frac{1}{\rho} \sum_{\zeta} f_\zeta \, e_{\zeta,x} \,, \tag{3.61}$$

$$v = \frac{1}{\rho} \sum_{\zeta} f_\zeta \, e_{\zeta,y} \,, \tag{3.62}$$

$$w = \frac{1}{\rho} \sum_{\zeta} f_\zeta \, e_{\zeta,z} \,. \tag{3.63}$$

Implementation: L222–238

```
// 0a. Density: rho = Sum f_alpha
double rho = 0.0;
for (int i = 0; i < 27; i++) rho += f_in[i];

// 0b. Momentum: j = Sum f_alpha * e_alpha
double jx = 0.0, jy = 0.0, jz = 0.0;
for (int i = 0; i < 27; i++) {
    jx += f_in[i] * GILBM_e[i][0];
    jy += f_in[i] * GILBM_e[i][1];
    jz += f_in[i] * GILBM_e[i][2];
}

// 0c. Half-force corrected velocity (Guo 2002, Thesis Eq.3.35)
double inv_rho = 1.0 / rho;
double u[3];
u[0] = jx * inv_rho + 0.5 * Fx * inv_rho * delta_t;
u[1] = jy * inv_rho + 0.5 * Fy * inv_rho * delta_t;
u[2] = jz * inv_rho + 0.5 * Fz * inv_rho * delta_t;
```

Note on Body Force (Thesis §3.2.1, p. 48)

The half-force velocity correction $\tilde{u} = u + F\Delta t/(2\rho)$ corresponds to Thesis Eq. 3.35–3.36: $\rho\tilde{u} = \rho u + \Delta t F/2$. This modified velocity $\tilde{u}$ enters the Chimera transform and all equilibrium computations. Combined with the sign flip of first-order central moments (L699–701) before the backward Chimera, this implements the Strang-splitting body force scheme.

3 Stage 1: Forward Chimera Transform (Eq. 3.64)

Thesis Eq. 3.64 / Geier 2015 Appendix J, Eq. J.4–J.12

The transformation path from PDFs to cumulants via central moments:

$$f_\zeta \rightleftharpoons \kappa_{\alpha\beta\gamma}(f_\zeta, (i,j,k)_\zeta, w_\zeta, \delta\rho, \boldsymbol{u}, c) \rightleftharpoons C_{\alpha\beta\gamma}(\kappa_{\alpha\beta\gamma}, \rho, \delta\rho)\ [= c_{\alpha\beta\gamma}\rho]\,. \tag{3.64}$$

The forward Chimera operates as three successive 1D transforms in the order $z \to y \to x$. For each direction, triplets $\{a, b, c\}$ (negative, zero, positive component) are combined:

$$m'_a = m_a + m_b + m_c\,, \tag{J.4: sum}$$
$$m'_b = (m_c - m_a) - (m'_a + K) \cdot u_{\text{dir}}\,, \tag{J.5: 1st moment}$$
$$m'_c = (m_a + m_c) - 2(m_c - m_a) \cdot u_{\text{dir}} + u_{\text{dir}}^2 \cdot (m'_a + K)\,. \tag{J.6: 2nd moment}$$

Here K is the well-conditioning correction constant.

Implementation: L759–780

```
__device__ static void _cum_forward_chimera(
    double m[27], const double u[3])
{
    for (int dir = 2; dir >= 0; dir--) {    // z(2) -> y(1) -> x(0)
        int base = (2 - dir) * 9;
        for (int j = 0; j < 9; j++) {
            int p = base + j;
            int a = CUM_IDX[p][0];
            int b = CUM_IDX[p][1];
            int c = CUM_IDX[p][2];
            double k = CUM_K[p];

            double sum  = m[a] + m[c];
            double diff = m[c] - m[a];

            m[a] = m[a] + m[b] + m[c];              // kappa_0 (sum)
            m[b] = diff - (m[a] + k) * u[dir];      // kappa_1
            m[c] = sum - 2.0*diff*u[dir]
                       + u[dir]*u[dir]*(m[a] + k);  // kappa_2
        }
    }
}
```

Chimera Index Tables

`CUM_IDX[27][3]` and `CUM_K[27]` are defined in `cumulant_constants.h`. After the full forward sweep, array position `m[p]` holds the central moment $\kappa_{\alpha\beta\gamma}$. The index aliases (`I_aaa`, `I_caa`, etc.) provide named access, e.g. `m[I_caa]` $= \kappa_{200}$.

4 Stage 2: Central Moments → Cumulants

For orders $\tilde{o} \leq 3$, cumulants equal central moments: $C_{\alpha\beta\gamma} = \kappa_{\alpha\beta\gamma}$. For orders 4–6, products of lower-order cumulants must be subtracted (Geier 2015, Appendix J, Eq. J.16–J.19).

4.1 4th-Order Off-Diagonal (Eq. J.16)

Geier 2015, Eq. J.16

$$
\begin{aligned}
C_{211} &= \kappa_{211} - \frac{1}{\rho}\left[(\kappa_{200} + \tfrac{1}{3})\kappa_{011} + 2\kappa_{110}\kappa_{101}\right], \\
C_{121} &= \kappa_{121} - \frac{1}{\rho}\left[(\kappa_{020} + \tfrac{1}{3})\kappa_{101} + 2\kappa_{110}\kappa_{011}\right], \\
C_{112} &= \kappa_{112} - \frac{1}{\rho}\left[(\kappa_{002} + \tfrac{1}{3})\kappa_{110} + 2\kappa_{101}\kappa_{011}\right]. \qquad \text{(J.16)}
\end{aligned}
$$

Implementation: L267–272

```
double CUMcbb = m[I_cbb] - ((m[I_caa] + 1.0/3.0) * m[I_abb]
                + 2.0 * m[I_bba] * m[I_bab]) * inv_rho;
double CUMbcb = m[I_bcb] - ((m[I_aca] + 1.0/3.0) * m[I_bab]
                + 2.0 * m[I_bba] * m[I_abb]) * inv_rho;
double CUMbbc = m[I_bbc] - ((m[I_aac] + 1.0/3.0) * m[I_bba]
                + 2.0 * m[I_bab] * m[I_abb]) * inv_rho;
```

These are computed in **both** AO and WP modes (no preprocessor guard), because:

1. WP mode: needed for general ω_8 formula (Eq. 3.86–3.88)
2. AO mode: needed for standard relaxation + back-conversion (Eq. J.16 inverse)

4.2 4th-Order Diagonal (Eq. J.17) and Higher Orders

The 4th-order diagonal (Eq. J.17, L275–283), 5th-order (Eq. J.18, L286–297), and 6th-order (Eq. J.19, L303–321) conversions follow the same pattern: subtract products of lower-order cumulants divided by ρ. These are standard and unchanged from the thesis.

5 Stage 3: Relaxation (Collision in Cumulant Space)

5.1 Eq. 3.65–3.66 — Relaxation Rates

Thesis Eq. 3.65–3.66

The relaxation rate's triple Greek index is replaced by a single index:

$$\omega_{\alpha\beta\gamma} \;\to\; \omega_C\,, \qquad C = \{1, \ldots, 10\}\,. \tag{3.65}$$

The first relaxation rate is linked to the shear viscosity:

$$\omega_1 = \omega = \frac{2}{6\nu + 1}\,, \tag{3.66}$$

defined by Eq. 3.18: $\tau = 3\nu + 1/2$, so $\omega_1 = 1/\tau$.

Implementation: L215

```
// omega_tau is the relaxation TIME tau = 3*nu/dt + 0.5
const double omega = 1.0 / omega_tau;    // w1 = 1/tau (shear relaxation RATE)
```

GILBM Convention

In the GILBM framework, $\tau = 3\nu/\Delta t_{\text{global}} + 0.5$ where $\Delta t_{\text{global}} \neq 1$. The code receives τ as `omega_tau` and converts to $\omega_1 = 1/\tau$ at L215.

5.2 Relaxation Rate Definitions (ω_2, ω_6–ω_{10})

Thesis §3.2.3: Relaxation Rate Assignment

- ω_2: bulk viscosity (free parameter, linked to ν_b)
- $\omega_3, \omega_4, \omega_5$: computed from ω_1, ω_2 in WP mode (Eq. 3.103–3.105)
- ω_6: 4th-order deviatoric relaxation (Eq. 3.83–3.85, deviatoric linear combinations)
- ω_7: 4th-order trace relaxation (Eq. 3.83–3.85, trace linear combination)
- ω_8: 4th-order off-diagonal relaxation (Eq. 3.86–3.88)
- ω_9: 5th-order relaxation (Eq. 3.90)
- ω_{10}: 6th-order relaxation (Eq. 3.91)

Standard WP/AO assignment: $\omega_{C\geq 6} = 1$. This implementation retains all as parameters.

Implementation: L345–382 — Macro-controlled ω definitions

```
// omega2: bulk viscosity
#ifdef CUM_OMEGA2
    const double omega2 = CUM_OMEGA2;
#else
    const double omega2 = 0.5;    // fallback default
#endif

// omega6: 4th-order deviatoric (Eq.3.83-3.85)
#ifdef CUM_OMEGA6
    const double omega6  = CUM_OMEGA6;
#else
    const double omega6  = 1.0;    // default: thesis WP/AO
#endif

// omega7: 4th-order trace (Eq.3.83-3.85)
#ifdef CUM_OMEGA7
    const double omega7  = CUM_OMEGA7;
#else
    const double omega7  = 1.0;    // default: thesis WP/AO
#endif

// omega8: 4th-order off-diagonal (Eq.3.86-3.88)
#ifdef CUM_OMEGA8
    const double omega8  = CUM_OMEGA8;
#else
    const double omega8  = 1.0;    // default: thesis WP/AO
#endif

// omega9: 5th order;  omega10: 6th order
#ifdef CUM_OMEGA9
    const double omega9  = CUM_OMEGA9;
#else
    const double omega9  = 1.0;
#endif
#ifdef CUM_OMEGA10
    const double omega10 = CUM_OMEGA10;
#else
    const double omega10 = 1.0;
```

```
#endif
```

Override any rate by defining `CUM_OMEGA6` etc. in `variables.h` before including `cumulant_collision.h`.

5.3 Eq. 3.67–3.68 — Conservation (0th, 1st Order)

Thesis Eq. 3.67–3.68

The zeroth and first-order cumulants are conserved:

$$C^*_{000} = C_{000}\,, \tag{3.67}$$

$$C^*_{100} = C_{100}\,, \quad C^*_{010} = C_{010}\,, \quad C^*_{001} = C_{001}\,. \tag{3.68}$$

Implementation

These are implicitly conserved: the code does not modify `m[I_aaa]` (density), `m[I_baa]`, `m[I_aba]`, `m[I_aab]` (momenta) during relaxation. The sign flip at L699–701 is for the Strang-splitting body force, not relaxation.

5.4 Eq. 3.69 — Off-Diagonal 2nd Order

Thesis Eq. 3.69

The three off-diagonal second-order cumulants undergo simple relaxation with ω_1:

$$C^*_{110} = (1-\omega_1)\,C_{110}\,, \quad C^*_{101} = (1-\omega_1)\,C_{101}\,, \quad C^*_{011} = (1-\omega_1)\,C_{011}\,. \tag{3.69}$$

Implementation: L478–480

```
m[I_abb] *= (1.0 - omega);  // C011 (yz)
m[I_bab] *= (1.0 - omega);  // C101 (xz)
m[I_bba] *= (1.0 - omega);  // C110 (xy)
```

Pre-relaxation values saved for WP mode

In WP mode, the **pre-relaxation** off-diagonal 2nd-order values are saved at L474–476 *before* the relaxation at L478–480. These saved values are needed for the off-diagonal 4th-order equilibrium (Eq. 3.86–3.89):

```
const double saved_C011 = m[I_abb];  // pre-relaxation C_{yz}
const double saved_C101 = m[I_bab];  // pre-relaxation C_{xz}
const double saved_C110 = m[I_bba];  // pre-relaxation C_{xy}
```

5.5 Eq. 3.70–3.75 — Diagonal 2nd Order with Galilean Correction

Thesis Eq. 3.70–3.75

The three diagonal second-order cumulants are decomposed into deviatoric and trace modes, with Galilean correction terms involving velocity derivatives:

Velocity derivatives (Eq. 3.73–3.75):

$$\partial_x u = \big[\omega_1(-2C_{200} + C_{020} + C_{002}) - \omega_2(C_{200} + C_{020} + C_{002} - \kappa_{000})\big]/(2\rho)\,, \tag{3.73}$$
$$\partial_y v = \big[\omega_1(C_{200} - 2C_{020} + C_{002}) - \omega_2(C_{200} + C_{020} + C_{002} - \kappa_{000})\big]/(2\rho)\,, \tag{3.74}$$
$$\partial_z w = \big[\omega_1(C_{200} + C_{020} - 2C_{002}) - \omega_2(C_{200} + C_{020} + C_{002} - \kappa_{000})\big]/(2\rho)\,. \tag{3.75}$$

Galilean correction terms (Eq. 3.70–3.72):

$$G_{\text{dev1}} = -3\rho(1-\omega_1/2)(\partial_x u \cdot u^2 - \partial_y v \cdot v^2)\,, \tag{3.70 part}$$
$$G_{\text{dev2}} = -3\rho(1-\omega_1/2)(\partial_x u \cdot u^2 - \partial_z w \cdot w^2)\,, \tag{3.71 part}$$
$$G_{\text{trace}} = -3\rho(1-\omega_2/2)(\partial_x u \cdot u^2 + \partial_y v \cdot v^2 + \partial_z w \cdot w^2)\,. \tag{3.72 part}$$

Relaxation:

$$\begin{aligned}(\text{dev1})^* &= (1-\omega_1)\cdot \text{dev1} + G_{\text{dev1}}\,,\\ (\text{dev2})^* &= (1-\omega_1)\cdot \text{dev2} + G_{\text{dev2}}\,,\\ (\text{trace})^* &= \omega_2 \cdot \kappa_{000} + (1-\omega_2)\cdot \text{trace} + G_{\text{trace}}\,,\end{aligned} \tag{3.70–3.72}$$

where $\text{dev1} = C_{200} - C_{020}$, $\text{dev2} = C_{200} - C_{002}$, $\text{trace} = C_{200} + C_{020} + C_{002}$.

Implementation: L442–468, L511–513

```
// Decompose into orthogonal modes
double mxxPyyPzz = m[I_caa] + m[I_aca] + m[I_aac];  // trace
double mxxMyy    = m[I_caa] - m[I_aca];              // deviatoric 1
double mxxMzz    = m[I_caa] - m[I_aac];              // deviatoric 2

// Eq.3.73-3.75: velocity derivatives from PRE-relaxation cumulants
double Dxux = -0.5*omega*inv_rho*(2.0*m[I_caa] - m[I_aca] - m[I_aac])
              - 0.5*omega2*inv_rho*(mxxPyyPzz - drho);
double Dyuy = Dxux + 1.5*omega*inv_rho*(m[I_caa] - m[I_aca]);
double Dzuz = Dxux + 1.5*omega*inv_rho*(m[I_caa] - m[I_aac]);

// Eq.3.70-3.72: Galilean correction terms
double GalCorr_dev1  = -3.0*rho*(1.0-0.5*omega)
                       *(Dxux*u[0]*u[0] - Dyuy*u[1]*u[1]);
double GalCorr_dev2  = -3.0*rho*(1.0-0.5*omega)
                       *(Dxux*u[0]*u[0] - Dzuz*u[2]*u[2]);
double GalCorr_trace = -3.0*rho*(1.0-0.5*omega2)
                       *(Dxux*u[0]*u[0] + Dyuy*u[1]*u[1] + Dzuz*u[2]*u[2]);

// Relax with Galilean correction
mxxMyy    = (1.0-omega)*mxxMyy + GalCorr_dev1;
mxxMzz    = (1.0-omega)*mxxMzz + GalCorr_dev2;
mxxPyyPzz = omega2*m[I_aaa] + (1.0-omega2)*mxxPyyPzz + GalCorr_trace;

// Reconstruct individual 2nd-order moments (L511-513)
m[I_caa] = ( mxxMyy + mxxMzz + mxxPyyPzz) / 3.0;
m[I_aca] = (-2.0*mxxMyy + mxxMzz + mxxPyyPzz) / 3.0;
m[I_aac] = ( mxxMyy - 2.0*mxxMzz + mxxPyyPzz) / 3.0;
```

Efficient computation of Eq. 3.73–3.75

The code computes $\partial_x u$ first, then obtains $\partial_y v$ and $\partial_z w$ by adding the appropriate ω_1 correction terms. This is algebraically equivalent to Eq. 3.73–3.75 (verified by direct expansion) but saves two multiplications.

Important: The velocity derivatives `Dxux`, `Dyuy`, `Dzuz` are used again later in the 4th-order diagonal equilibrium (Eq. 3.83–3.85). They are computed from **pre-relaxation** cumulants, which is correct per the thesis derivation.

5.6 Eq. 3.76–3.82 — Third-Order Relaxation

Thesis Eq. 3.76–3.82

The seven third-order cumulants are grouped into three symmetric/antisymmetric pairs plus one standalone:

$$C^*_{102} = [(1-\tilde{\omega}_{3,1})(C_{120}+C_{102}) - (1-\tilde{\omega}_{4,1})(C_{120}-C_{102})]/2\,, \tag{3.76}$$
$$C^*_{120} = [(1-\tilde{\omega}_{3,1})(C_{120}+C_{102}) + (1-\tilde{\omega}_{4,1})(C_{120}-C_{102})]/2\,, \tag{3.77}$$
$$C^*_{012} = [(1-\tilde{\omega}_{3,2})(C_{210}+C_{012}) - (1-\tilde{\omega}_{4,2})(C_{210}-C_{012})]/2\,, \tag{3.78}$$
$$C^*_{210} = [(1-\tilde{\omega}_{3,2})(C_{210}+C_{012}) + (1-\tilde{\omega}_{4,2})(C_{210}-C_{012})]/2\,, \tag{3.79}$$
$$C^*_{021} = [(1-\tilde{\omega}_{3,3})(C_{201}+C_{021}) - (1-\tilde{\omega}_{4,3})(C_{201}-C_{021})]/2\,, \tag{3.80}$$
$$C^*_{201} = [(1-\tilde{\omega}_{3,3})(C_{201}+C_{021}) + (1-\tilde{\omega}_{4,3})(C_{201}-C_{021})]/2\,, \tag{3.81}$$
$$C^*_{111} = (1-\tilde{\omega}_5)\,C_{111}\,. \tag{3.82}$$

The tilde notation $\tilde{\omega}$ refers to the lambda-limited rates (Eq. 3.111–3.114). In AO mode, all $\tilde{\omega} = 1$, so all 3rd-order post-collision cumulants $= 0$.

Implementation: L483–521 (WP path shown)

```
// Symmetric / antisymmetric decomposition
double mxxyPyzz = m[I_cba] + m[I_abc];   // C210 + C012
double mxxyMyzz = m[I_cba] - m[I_abc];   // C210 - C012
double mxxzPyyz = m[I_cab] + m[I_acb];   // C201 + C021
double mxxzMyyz = m[I_cab] - m[I_acb];   // C201 - C021
double mxyyPxzz = m[I_bca] + m[I_bac];   // C120 + C102
double mxyyMxzz = m[I_bca] - m[I_bac];   // C120 - C102

#if USE_WP_CUMULANT
    // WP: each pair has its OWN lambda-limited rate
    m[I_bbb]  *= (1.0 - omega5_lim);     // C111 (Eq.3.82/3.114)
    mxxyPyzz  *= (1.0 - omega3_2);       // (C210+C012) sym (Eq.3.78-79/3.112)
    mxxyMyzz  *= (1.0 - omega4_2);       // (C210-C012) antisym
    mxxzPyyz  *= (1.0 - omega3_3);       // (C201+C021) sym (Eq.3.80-81/3.113)
    mxxzMyyz  *= (1.0 - omega4_3);       // (C201-C021) antisym
    mxyyPxzz  *= (1.0 - omega3_1);       // (C120+C102) sym (Eq.3.76-77/3.111)
    mxyyMxzz  *= (1.0 - omega4_1);       // (C120-C102) antisym
#else
    // AO: all omega = 1 -> (1-omega) = 0 -> full relaxation to zero
    m[I_bbb]  = 0.0;
    mxxyPyzz  = 0.0; mxxyMyzz  = 0.0;
    mxxzPyyz  = 0.0; mxxzMyyz  = 0.0;
    mxyyPxzz  = 0.0; mxyyMxzz  = 0.0;
#endif

// Reconstruct individual 3rd-order moments (L516-521)
m[I_cba] = ( mxxyMyzz + mxxyPyzz) * 0.5;   // C210
m[I_abc] = (-mxxyMyzz + mxxyPyzz) * 0.5;   // C012
m[I_cab] = ( mxxzMyyz + mxxzPyyz) * 0.5;   // C201
m[I_acb] = (-mxxzMyyz + mxxzPyyz) * 0.5;   // C021
m[I_bca] = ( mxyyMxzz + mxyyPxzz) * 0.5;   // C120
m[I_bac] = (-mxyyMxzz + mxyyPxzz) * 0.5;   // C102
```

5.7 Eq. 3.83–3.85 — Fourth-Order Diagonal (Thesis Original Form)

Thesis Eq. 3.83–3.85 — FULL unsimplified form

The three diagonal fourth-order post-collision cumulants involve parameter A, velocity derivatives $\partial_x u$, $\partial_y v$, $\partial_z w$, and three relaxation rates: ω_6 (deviatoric modes) and ω_7 (trace mode).
Eq. 3.83 (C^*_{220}):

$$\begin{aligned} C^*_{220} = & \tfrac{2}{3}(1/\omega_1 - \tfrac{1}{2})A\rho(\partial_x u - 2\partial_y v + \partial_z w) + (1-\omega_6)(C_{220} - 2C_{202} + C_{022}) \\ & + \tfrac{2}{3}(1/\omega_1 - \tfrac{1}{2})A\rho(\partial_x u + \partial_y v - 2\partial_z w) + (1-\omega_6)(C_{220} + C_{202} - 2C_{022}) \\ & - \tfrac{4}{3}(1/\omega_1 - \tfrac{1}{2})A\rho(\partial_x u + \partial_y v + \partial_z w) + (1-\omega_7)(C_{220} + C_{202} + C_{022}) \\ & \text{all divided by } 3\,. \end{aligned} \tag{3.83}$$

Eq. 3.84 (C^*_{202}):

$$\begin{aligned} C^*_{202} = & \big[-\big(\tfrac{2}{3}(1/\omega_1 - \tfrac{1}{2})A\rho(\partial_x u - 2\partial_y v + \partial_z w) + (1-\omega_6)(C_{220} - 2C_{202} + C_{022})\big) \\ & + \big(-\tfrac{4}{3}(1/\omega_1 - \tfrac{1}{2})A\rho(\partial_x u + \partial_y v + \partial_z w) + (1-\omega_7)(C_{220} + C_{202} + C_{022})\big)\big]/3\,. \end{aligned} \tag{3.84}$$

Eq. 3.85 (C^*_{022}):

$$\begin{aligned} C^*_{022} = & \big[-\big(\tfrac{2}{3}(1/\omega_1 - \tfrac{1}{2})A\rho(\partial_x u + \partial_y v - 2\partial_z w) + (1-\omega_6)(C_{220} + C_{202} - 2C_{022})\big) \\ & + \big(-\tfrac{4}{3}(1/\omega_1 - \tfrac{1}{2})A\rho(\partial_x u + \partial_y v + \partial_z w) + (1-\omega_7)(C_{220} + C_{202} + C_{022})\big)\big]/3\,. \end{aligned} \tag{3.85}$$

Structure: each post-collision cumulant is reconstructed from three orthogonal modes:

- **dev1** $= C_{220} - 2C_{202} + C_{022}$ (deviatoric, relaxed with ω_6)
- **dev2** $= C_{220} + C_{202} - 2C_{022}$ (deviatoric, relaxed with ω_6)
- **trace** $= C_{220} + C_{202} + C_{022}$ (isotropic, relaxed with ω_7)

Each mode satisfies:

$$\text{mode}^* = c^{\text{eq}}_{\text{mode}} + (1 - \omega_{6 \text{ or } 7}) \cdot \text{mode}_{\text{pre}}$$

where the equilibrium terms use $(1/\omega_1 - 1/2) \cdot A \cdot \rho \cdot$ (velocity derivatives).

Implementation: L524–564 (WP mode)

```
// WP: 4th-order DIAGONAL collision -- Thesis Eq. 3.83-3.85 (original form)
{
    // Velocity-derivative equilibrium coefficient: (1/w1 - 1/2)*A*rho
    const double vel_eq_A = (1.0/omega - 0.5) * coeff_A * rho;

    // Eq. 3.83-3.85 equilibrium terms for each orthogonal mode:
    //   dev1:  2/3 * vel_eq_A * (dxu - 2*dyv + dzw)
    //   dev2:  2/3 * vel_eq_A * (dxu + dyv - 2*dzw)
    //   trace: -4/3 * vel_eq_A * (dxu + dyv + dzw)
    const double eq4_dev1  =  2.0/3.0 * vel_eq_A * (Dxux - 2.0*Dyuy + Dzuz);
    const double eq4_dev2  =  2.0/3.0 * vel_eq_A * (Dxux + Dyuy - 2.0*Dzuz);
    const double eq4_trace = -4.0/3.0 * vel_eq_A * (Dxux + Dyuy + Dzuz);

    // Pre-collision deviatoric/trace decomposition
    double cum4_dev1  = CUMcca - 2.0*CUMcac + CUMacc;
    double cum4_dev2  = CUMcca + CUMcac - 2.0*CUMacc;
    double cum4_trace = CUMcca + CUMcac + CUMacc;

    // Thesis Eq. 3.83-3.85 (original form, NO simplification):
    //   mode* = eq_term + (1 - omega) * mode_pre
    cum4_dev1  = eq4_dev1  + (1.0 - omega6) * cum4_dev1;    // deviatoric (w6)
    cum4_dev2  = eq4_dev2  + (1.0 - omega6) * cum4_dev2;    // deviatoric (w6)
    cum4_trace = eq4_trace + (1.0 - omega7) * cum4_trace;   // trace (w7)
```

```

    // Reconstruct individual 4th-order diagonal cumulants
    CUMcca = (cum4_dev1 + cum4_dev2 + cum4_trace) / 3.0;
    CUMcac = (cum4_trace - cum4_dev1) / 3.0;
    CUMacc = (cum4_trace - cum4_dev2) / 3.0;
}
```

Why velocity-derivative form, not [GR22

product form?] An alternative formulation from [GR22] Eq. 17–18 expresses the 4th-order equilibria using products of post-relaxation 2nd-order cumulants with parameters A and B:

$$C_{220}^{\text{eq}} = (A \cdot \sigma_{xx}\sigma_{yy} + B \cdot \tau_{xy}^2) \cdot \rho^{-1}$$

However, this product form is **only equivalent to the thesis velocity-derivative form when** $\omega_6 = \omega_7 = 1$. For general $\omega_6 \neq 1$ or $\omega_7 \neq 1$, the product form produces different (incorrect) results because the equilibrium and the non-equilibrium terms couple differently. The velocity-derivative form (Eq. 3.83–3.85) is the **fundamental, general** formulation that works for **any** ω_6, ω_7 values.

5.8 Eq. 3.86–3.88 — Fourth-Order Off-Diagonal (Thesis Original Form)

Thesis Eq. 3.86–3.88 — FULL unsimplified form

The three off-diagonal fourth-order cumulants involve parameter B, velocity derivatives from Eq. 3.89, and relaxation rate ω_8:

$$C_{211}^* = -\tfrac{1}{3}(\omega_1/2 - 1)\,\omega_8\,B\rho\,(\partial_y w + \partial_z v) + (1 - \omega_8)\,C_{211}\,, \tag{3.86}$$
$$C_{121}^* = -\tfrac{1}{3}(\omega_1/2 - 1)\,\omega_8\,B\rho\,(\partial_x w + \partial_z u) + (1 - \omega_8)\,C_{121}\,, \tag{3.87}$$
$$C_{112}^* = -\tfrac{1}{3}(\omega_1/2 - 1)\,\omega_8\,B\rho\,(\partial_x v + \partial_y u) + (1 - \omega_8)\,C_{112}\,. \tag{3.88}$$

Each is a standard relaxation: $C^* = \omega_8 \cdot C^{\text{eq}} + (1 - \omega_8) \cdot C$, where the equilibrium involves velocity derivatives and parameter B. When $\omega_8 = 1$: the $(1-\omega_8)\cdot C$ term vanishes, recovering Eq. 3.119–3.121.

Thesis Eq. 3.89 — Velocity Derivatives from Off-Diagonal Cumulants

$$\begin{aligned}
\partial_y w + \partial_z v &= -3\omega_1/\rho \cdot C_{011}\,,\\
\partial_x w + \partial_z u &= -3\omega_1/\rho \cdot C_{101}\,,\\
\partial_x v + \partial_y u &= -3\omega_1/\rho \cdot C_{110}\,.
\end{aligned} \tag{3.89, corrected}$$

ERRATA: Thesis Eq. 3.89 has a typographic error

The thesis text prints the three velocity derivative relations as:

$$\begin{aligned}
\partial_y w + \partial_z v &= -3\omega_1/\rho \cdot C_{\mathbf{110}}\,,\\
\partial_x w + \partial_z u &= -3\omega_1/\rho \cdot C_{101}\,,\\
\partial_x v + \partial_y u &= -3\omega_1/\rho \cdot C_{\mathbf{011}}\,.
\end{aligned}$$

The subscripts C_{110} and C_{011} are **swapped** in the first and third relations.
Verification via Chapman–Enskog analysis:

- C_{011} is the yz-stress cumulant → corresponds to $\partial_y w + \partial_z v$ (yz shear)
- C_{110} is the xy-stress cumulant → corresponds to $\partial_x v + \partial_y u$ (xy shear)

The code uses the **corrected** mapping (matching walberla's implementation).

Implementation: L569–584 (WP mode)

```
// Step 1: Velocity derivatives from Eq. 3.89 (CORRECTED subscripts)
const double dywPdzv = -3.0 * omega * inv_rho * saved_C011;  // dy w + dz v
const double dxwPdzu = -3.0 * omega * inv_rho * saved_C101;  // dx w + dz u
const double dxvPdyu = -3.0 * omega * inv_rho * saved_C110;  // dx v + dy u

// Step 2: General collision Eq. 3.86-3.88 with omega8 retained
//   When omega8=1: (1-omega8)*C = 0, reduces to Eq. 3.119-3.121
const double eq_offdiag_coeff = -1.0/3.0 * (omega*0.5 - 1.0)
                                * omega8 * coeff_B * rho;
const double wp_C211_star = eq_offdiag_coeff * dywPdzv
                          + (1.0 - omega8) * CUMcbb;  // Eq.3.86
const double wp_C121_star = eq_offdiag_coeff * dxwPdzu
                          + (1.0 - omega8) * CUMbcb;  // Eq.3.87
const double wp_C112_star = eq_offdiag_coeff * dxvPdyu
                          + (1.0 - omega8) * CUMbbc;  // Eq.3.88
```

Algebraic verification

Substituting Eq. 3.89 into the equilibrium part of Eq. 3.86:

$$C_{211}^{\text{eq}} = -\tfrac{1}{3}(\omega_1/2 - 1)\,\omega_8\,B\rho \cdot (-3\omega_1/\rho \cdot C_{011}) = \omega_8 \cdot (\omega_1/2 - 1) \cdot \omega_1 \cdot B \cdot C_{011}$$

The code computes: `eq_offdiag_coeff` $= -\frac{1}{3}(\omega/2 - 1) \cdot \omega_8 \cdot B \cdot \rho$, then multiplies by `dywPdzv` $= -3\omega/\rho \cdot C_{011}$:

$$-\tfrac{1}{3}(\omega/2 - 1) \cdot \omega_8 \cdot B \cdot \rho \cdot (-3\omega/\rho \cdot C_{011}) = \omega_8 \cdot (\omega/2 - 1) \cdot \omega \cdot B \cdot C_{011} \quad \checkmark$$

5.9 AO Mode: Fourth-Order Relaxation

Implementation: L586–605 (AO mode)

In AO mode, the equilibria are zero ($A = B = 0$), but the code still uses $\omega_6/\omega_7/\omega_8$ so they can be tuned independently:

```
#else  // AO mode
    // Diagonal: omega6 (deviatoric), omega7 (trace), eq=0
    {
        double cum4_dev1  = CUMcca - 2.0*CUMcac + CUMacc;
        double cum4_dev2  = CUMcca + CUMcac - 2.0*CUMacc;
        double cum4_trace = CUMcca + CUMcac + CUMacc;

        cum4_dev1  *= (1.0 - omega6);   // deviatoric (w6)
        cum4_dev2  *= (1.0 - omega6);   // deviatoric (w6)
        cum4_trace *= (1.0 - omega7);   // trace (w7)

        CUMcca = (cum4_dev1 + cum4_dev2 + cum4_trace) / 3.0;
        CUMcac = (cum4_trace - cum4_dev1) / 3.0;
        CUMacc = (cum4_trace - cum4_dev2) / 3.0;
    }
    // Off-diagonal: omega8, eq=0 since A=B=0
    CUMbbc *= (1.0 - omega8);
    CUMbcb *= (1.0 - omega8);
    CUMcbb *= (1.0 - omega8);
#endif
```

When $\omega_6 = \omega_7 = \omega_8 = 1$ (default), all 4th-order cumulants are fully damped to zero, matching Eq. 3.102.

5.10 Eq. 3.90–3.91 — Fifth and Sixth Order

Thesis Eq. 3.90–3.91

$$C^*_{221} = (1-\omega_9)\, C_{221}\,, \quad C^*_{212} = (1-\omega_9)\, C_{212}\,, \quad C^*_{122} = (1-\omega_9)\, C_{122}\,, \tag{3.90}$$

$$C^*_{222} = (1-\omega_{10})\, C_{222}\,. \tag{3.91}$$

Implementation: L608–614

```
// 5th order: equilibria = 0
CUMbcc *= (1.0 - omega9);     // C_122
CUMcbc *= (1.0 - omega9);     // C_212
CUMccb *= (1.0 - omega9);     // C_221

// 6th order: equilibria = 0
CUMccc *= (1.0 - omega10);    // C_222
```

6 Employed Cumulant Variants (§3.2.3)

6.1 Eq. 3.92–3.102 — All-One (C-AO) Variant

Thesis Eq. 3.92–3.102

In the AO variant, all free relaxation rates are set to unity:

$$\omega_{C\geq 2} = 1\,, \qquad \omega_1 = \omega = \frac{2}{6\nu+1}\,, \tag{3.92}$$

and $A = B = 0$. All post-collision cumulants of order $\tilde{o} > 2$ are reset to zero:

$$C^*_{\tilde{o}>2} = 0\,. \tag{3.102}$$

Implementation: `USE_WP_CUMULANT=0`

When `USE_WP_CUMULANT=0`, the AO path is activated:

- L500–507: All 3rd-order post-collision cumulants set to 0
- L589–605: All 4th-order relaxed with $\omega_6/\omega_7/\omega_8$ toward 0 equilibrium
- L608–614: 5th/6th order with ω_9/ω_{10}

With default $\omega_{6\ldots10} = 1$: all $\tilde{o} > 2$ cumulants become zero, exactly matching Eq. 3.102. With custom $\omega_6 < 1$: partial damping retained (extended AO).

6.2 Eq. 3.103–3.107 — Parameterized (C-P/WP) Variant

Thesis Eq. 3.103–3.105

The parameterized relaxation rates are functions of $\omega = \omega_1$ and ω_2:

$$\omega_3 = \frac{8(\omega-2)(\omega_2(3\omega-1)-5\omega)}{8(5-2\omega)\omega+\omega_2(8+\omega(9\omega-26))}, \tag{3.103}$$

$$\omega_4 = \frac{8(\omega-2)(\omega+\omega_2(3\omega-7))}{\omega_2(56-42\omega+9\omega^2)-8\omega}, \tag{3.104}$$

$$\omega_5 = \frac{24(\omega-2)(4\omega^2+\omega\omega_2(18-13\omega)+\omega_2^2(2+\omega(6\omega-11)))}{16\omega^2(\omega-6)-2\omega\omega_2(216+5\omega(9\omega-46))+\omega_2^2(\omega(3\omega-10)(15\omega-28)-48)}. \tag{3.105}$$

Implementation: L91–119

```
__device__ static void _cum_wp_compute_omega345(
    const double w1, const double w2,
    double *w3, double *w4, double *w5)
{
    const double DEN_EPS = 1.0e-10;

    // Eq.3.103: w3  (denominator zeros at w1 ~= 2.46, -0.46 -> safe)
    double num3  = 8.0*(w1-2.0)*(w2*(3.0*w1-1.0) - 5.0*w1);
    double den3  = 8.0*(5.0-2.0*w1)*w1
                 + w2*(8.0 + w1*(9.0*w1-26.0));
    double w3_raw = (fabs(den3) > DEN_EPS) ? num3/den3 : 1.0;
    *w3 = _cum_wp_clamp_with_ao_fallback(w3_raw);

    // Eq.3.104: w4  (SINGULARITY at w1=14/9 ~ 1.5556 for w2=1)
    double num4  = 8.0*(w1-2.0)*(w1 + w2*(3.0*w1-7.0));
    double den4  = w2*(56.0-42.0*w1+9.0*w1*w1) - 8.0*w1;
    double w4_raw = (fabs(den4) > DEN_EPS) ? num4/den4 : 1.0;
    *w4 = _cum_wp_clamp_with_ao_fallback(w4_raw);

    // Eq.3.105: w5  (denominator zeros at w1 ~= -0.256 -> safe)
    double num5  = 24.0*(w1-2.0)*(4.0*w1*w1
                 + w1*w2*(18.0-13.0*w1)
                 + w2*w2*(2.0+w1*(6.0*w1-11.0)));
    double den5  = 16.0*w1*w1*(w1-6.0)
                 - 2.0*w1*w2*(216.0+5.0*w1*(9.0*w1-46.0))
                 + w2*w2*(w1*(3.0*w1-10.0)*(15.0*w1-28.0)-48.0);
    double w5_raw = (fabs(den5) > DEN_EPS) ? num5/den5 : 1.0;
    *w5 = _cum_wp_clamp_with_ao_fallback(w5_raw);
}
```

Singularity handling: If the raw value falls outside $[0, 2]$, the function `_cum_wp_clamp_with_ao_fallback` (L80–89) returns 1.0 (AO neutral).

Thesis Eq. 3.106–3.107

The scalar parameters for fourth-order equilibria:

$$A = \frac{4\omega_1^2+2\omega_1\omega_2(\omega_1-6)+\omega_2^2(\omega_1(10-3\omega_1)-4)}{(\omega_1-\omega_2)(\omega_2(2+3\omega_1)-8\omega_1)}, \tag{3.106}$$

$$B = \frac{4\omega_1\omega_2(9\omega_1-16)-4\omega_1^2-2\omega_2^2(2+9\omega_1(\omega_1-2))}{3(\omega_1-\omega_2)(\omega_2(2+3\omega_1)-8\omega_1)}. \tag{3.107}$$

Implementation: L140–165

```
__device__ static void _cum_wp_compute_AB(
    const double w1, const double w2,
```

```
    double *A, double *B)
{
    const double DEN_EPS = 1.0e-10;
    // Denominator: (w1-w2)*(w2*(2+3*w1)-8*w1)
    //    Zeros at w1=w2 and w1=2*w2/(8-3*w2)
    double denom = (w1-w2) * (w2*(2.0+3.0*w1) - 8.0*w1);

    if (fabs(denom) > DEN_EPS) {
        // Eq.3.106: A
        *A = (4.0*w1*w1 + 2.0*w1*w2*(w1-6.0)
            + w2*w2*(w1*(10.0-3.0*w1)-4.0)) / denom;
        // Eq.3.107: B
        *B = (4.0*w1*w2*(9.0*w1-16.0) - 4.0*w1*w1
            - 2.0*w2*w2*(2.0+9.0*w1*(w1-2.0)))
            / (3.0 * denom);
    } else {
        *A = 0.0;  *B = 0.0;    // fallback -> A0 (4th-order eq = 0)
    }
}
```

6.3 Eq. 3.108–3.110 — High-Re Limit Conservation

Thesis Eq. 3.108–3.110

In the low-viscosity (high-Re) limit $\nu \to 0$, $\omega \to 2$:

$$\lim_{\nu\to 0} \omega_{\{3;4;5\}}(\omega) \to 0\,. \tag{3.108}$$

This means the third-order cumulants are fully conserved:

$$C^*_{102} \to C_{102}\,,\; C^*_{120} \to C_{120}\,,\; \ldots \tag{3.109}$$

$$C^*_{021} \to C_{021}\,,\; C^*_{201} \to C_{201}\,,\; \ldots\; C^*_{111} \to C_{111}\,. \tag{3.110}$$

This hampers numerical stability at high Re, motivating the lambda-limiter (Eq. 3.111–3.115).

Implementation note

Eq. 3.108–3.110 are not directly implemented; they describe the limiting behavior. The lambda-limiter (next section) ensures that the effective rates $\tilde{\omega}$ never vanish completely, providing a minimum level of dissipation for stability.

6.4 Eq. 3.111–3.115 — Lambda Limiter (Regularization)

Thesis Eq. 3.111–3.115

To regularize the vanishing relaxation rates, a limiter is applied:

$$\tilde{\omega}_{3,1} = \omega_3 + \frac{(1-\omega_3)|C_{120}+C_{102}|}{\rho\lambda + |C_{120}+C_{102}|}\,, \quad \tilde{\omega}_{4,1} = \omega_4 + \frac{(1-\omega_4)|C_{120}-C_{102}|}{\rho\lambda + |C_{120}-C_{102}|}\,, \tag{3.111}$$

$$\tilde{\omega}_{3,2} = \omega_3 + \frac{(1-\omega_3)|C_{210}+C_{012}|}{\rho\lambda + |C_{210}+C_{012}|}\,, \quad \tilde{\omega}_{4,2} = \omega_4 + \frac{(1-\omega_4)|C_{210}-C_{012}|}{\rho\lambda + |C_{210}-C_{012}|}\,, \tag{3.112}$$

$$\tilde{\omega}_{3,3} = \omega_3 + \frac{(1-\omega_3)|C_{201}+C_{021}|}{\rho\lambda + |C_{201}+C_{021}|}\,, \quad \tilde{\omega}_{4,3} = \omega_4 + \frac{(1-\omega_4)|C_{201}-C_{021}|}{\rho\lambda + |C_{201}-C_{021}|}\,, \tag{3.113}$$

$$\tilde{\omega}_5 = \omega_5 + \frac{(1-\omega_5)|C_{111}|}{\rho\lambda + |C_{111}|}\,, \tag{3.114}$$

with $\lambda = \lambda_{\text{def}} = 10^{-2}$ (Eq. 3.115).

Implementation: L172–178 (helper) + L416–431 (application)

Helper function:

```
__device__ static double _cum_wp_limit(
    const double omega_base, const double C_mag,
    const double rho, const double lambda)
{
    double absC = fabs(C_mag);
    return omega_base + (1.0-omega_base)*absC / (rho*lambda + absC);
}
```

Application:

```
const double lam = CUM_LAMBDA;    // default 1e-2 (L46-48)

// Eq.3.111
double omega3_1 = _cum_wp_limit(omega3_base, C_120+C_102, rho, lam);
double omega4_1 = _cum_wp_limit(omega4_base, C_120-C_102, rho, lam);
// Eq.3.112
double omega3_2 = _cum_wp_limit(omega3_base, C_210+C_012, rho, lam);
double omega4_2 = _cum_wp_limit(omega4_base, C_210-C_012, rho, lam);
// Eq.3.113
double omega3_3 = _cum_wp_limit(omega3_base, C_201+C_021, rho, lam);
double omega4_3 = _cum_wp_limit(omega4_base, C_201-C_021, rho, lam);
// Eq.3.114
double omega5_lim = _cum_wp_limit(omega5_base, C_111, rho, lam);
```

6.5 Eq. 3.116–3.121 — Special Case $\omega_6 = \omega_7 = \omega_8 = 1$

Thesis Eq. 3.116–3.121

When all 4th-order rates are set to unity, Eq. 3.83–3.88 simplify to:
Diagonal (Eq. 3.116–3.118): the $(1-\omega_6)\cdot C$ and $(1-\omega_7)\cdot C$ terms vanish, leaving only the equilibrium:

$$C^*_{220} = (1 - 2/\omega_1)\, A\rho\, (\partial_y v + \partial_z w)/3\,, \tag{3.116}$$
$$C^*_{202} = (1 - 2/\omega_1)\, A\rho\, (\partial_x u + \partial_z w)/3\,, \tag{3.117}$$
$$C^*_{022} = (1 - 2/\omega_1)\, A\rho\, (\partial_x u + \partial_y v)/3\,. \tag{3.118}$$

Off-diagonal (Eq. 3.119–3.121): the $(1-\omega_8)\cdot C$ term vanishes:

$$C^*_{211} = (\omega_1/2 - 1)\, B\omega_1\, C_{011}\,, \tag{3.119}$$
$$C^*_{121} = (\omega_1/2 - 1)\, B\omega_1\, C_{101}\,, \tag{3.120}$$
$$C^*_{112} = (\omega_1/2 - 1)\, B\omega_1\, C_{110}\,. \tag{3.121}$$

These are special cases, not the implemented form

Eq. 3.116–3.121 are the **simplified** forms obtained by setting $\omega_6 = \omega_7 = \omega_8 = 1$. The code implements the **general** forms (Eq. 3.83–3.88) which retain $(1-\omega)\cdot C$ terms. When the default $\omega_6 = \omega_7 = \omega_8 = 1$ is used, the code automatically reduces to Eq. 3.116–3.121 since $(1-1)\cdot C = 0$.

7 Stage 4: Cumulants → Central Moments (Inverse)

The inverse of Stage 2 adds back the lower-order products (Eq. J.16–J.19 inverse).

7.1 4th-Order Off-Diagonal Inverse (Eq. J.16 inverse)

Eq. J.16 inverse

$$\begin{aligned}
\kappa_{211} &= C^*_{211} + \frac{1}{\rho}\left[(\kappa^*_{200} + \tfrac{1}{3})\kappa^*_{011} + 2\kappa^*_{110}\kappa^*_{101}\right], \\
\kappa_{121} &= C^*_{121} + \frac{1}{\rho}\left[(\kappa^*_{020} + \tfrac{1}{3})\kappa^*_{101} + 2\kappa^*_{110}\kappa^*_{011}\right], \\
\kappa_{112} &= C^*_{112} + \frac{1}{\rho}\left[(\kappa^*_{002} + \tfrac{1}{3})\kappa^*_{110} + 2\kappa^*_{101}\kappa^*_{011}\right]. \qquad \text{(J.16 inv)}
\end{aligned}$$

These low-order product terms are $O(\mathrm{Ma}^2)$, same order as C^*_{211} itself. Omitting them is a **bug** that would cause $O(\mathrm{Ma}^2)$ errors in the off-diagonal 4th-order central moments.

Implementation: L622–630 (WP) / L631–637 (AO)

```
#if USE_WP_CUMULANT
    // WP: cumulant -> central moment with full product terms
    m[I_cbb] = wp_C211_star + ((m[I_caa] + 1.0/3.0)*m[I_abb]
               + 2.0*m[I_bba]*m[I_bab]) * inv_rho;
    m[I_bcb] = wp_C121_star + ((m[I_aca] + 1.0/3.0)*m[I_bab]
               + 2.0*m[I_bba]*m[I_abb]) * inv_rho;
    m[I_bbc] = wp_C112_star + ((m[I_aac] + 1.0/3.0)*m[I_bba]
               + 2.0*m[I_bab]*m[I_abb]) * inv_rho;
#else
    // AO: same structure
    m[I_cbb] = CUMcbb + ((m[I_caa] + 1.0/3.0)*m[I_abb]
               + 2.0*m[I_bba]*m[I_bab]) * inv_rho;
    m[I_bcb] = CUMbcb + ((m[I_aca] + 1.0/3.0)*m[I_bab]
               + 2.0*m[I_bba]*m[I_abb]) * inv_rho;
    m[I_bbc] = CUMbbc + ((m[I_aac] + 1.0/3.0)*m[I_bba]
               + 2.0*m[I_bab]*m[I_abb]) * inv_rho;
#endif
```

Note: The values `m[I_caa]`, `m[I_abb]`, etc. are the **post-relaxation** 2nd-order central moments (already relaxed at L466–480, reconstructed at L511–513).

7.2 4th-Order Diagonal, 5th-Order, 6th-Order Inverse

These follow the same pattern as the forward conversion (adding products instead of subtracting). See L641–685 for the complete implementation matching Eq. J.17–J.19 inverse.

8 Stage 5: Backward Chimera and Body Force

8.1 Strang-Splitting Body Force (Thesis §3.2.1, p. 48)

Thesis §3.2.1

The body force is implicitly applied through the modified equilibrium velocity $\tilde{u} = u + F\Delta t/(2\rho)$ entering the collision (Eq. 3.35–3.36). The three first-order central moments must change sign prior to the back transformation to complete the Strang splitting.

Implementation: L699–701

```
// Sign flip for Strang-splitting body force
m[I_baa] = -m[I_baa];  // kappa*_100 (x-momentum)
m[I_aba] = -m[I_aba];  // kappa*_010 (y-momentum)
m[I_aab] = -m[I_aab];  // kappa*_001 (z-momentum)
```

No explicit Guo source term is used; the half-force velocity (Stage 0c) combined with this sign flip implements the full Strang-splitting scheme.

8.2 Backward Chimera ($x \to y \to z$)

Implementation: L787–811

```
__device__ static void _cum_backward_chimera(
    double m[27], const double u[3])
{
    for (int dir = 0; dir < 3; dir++) {   // x(0) -> y(1) -> z(2)
        int base = (2 - dir) * 9;
        for (int j = 0; j < 9; j++) {
            int p = base + j;
            int a = CUM_IDX[p][0];
            int b = CUM_IDX[p][1];
            int c = CUM_IDX[p][2];
            double k = CUM_K[p];

            double ma = ((m[c]-m[b])*0.5 + m[b]*u[dir]
                        + (m[a]+k)*(u[dir]*u[dir]-u[dir])*0.5);
            double mb = (m[a]-m[c]) - 2.0*m[b]*u[dir]
                        - (m[a]+k)*u[dir]*u[dir];
            double mc = ((m[c]+m[b])*0.5 + m[b]*u[dir]
                        + (m[a]+k)*(u[dir]*u[dir]+u[dir])*0.5);

            m[a] = ma;   m[b] = mb;   m[c] = mc;
        }
    }
}
```

After the backward Chimera, restore from well-conditioned form: `f_out[i] = m[i] + GILBM_W[i]` (L710–712).

9 Summary Table: Equation → Code Mapping

Equation	Description	File	Lines
3.59	Well-conditioning $f = f^c - w$	collision.h	240–244
3.60–3.63	$\delta\rho$, u, v, w + half-force	collision.h	222–238
3.64	Chimera path $f \rightleftharpoons \kappa \rightleftharpoons C$	collision.h	254, 707
3.65–3.66	$\omega_1 = 1/\tau$	collision.h	215
—	ω_2 (bulk), ω_6–ω_{10} macros	collision.h	345–382
3.67–3.68	Conservation (0th, 1st order)	collision.h	(implicit)
3.69	Off-diag 2nd order: $(1-\omega_1)C$	collision.h	478–480
3.70–3.72	Diag 2nd order + Galilean correction	collision.h	442–468
3.73–3.75	Velocity derivatives from 2nd-order cum.	collision.h	454–457
3.76–3.82	3rd order sym/antisym relaxation	collision.h	483–521
3.83–3.85	**4th order diagonal (general ω_6, ω_7)**	collision.h	**524–564**
3.86–3.88	**4th order off-diagonal (general ω_8)**	collision.h	**569–584**
3.89	**Vel. derivs from off-diag cum. (CORRECTED)**	collision.h	**575–577**

Equation	Description	File	Lines
3.90–3.91	5th & 6th order relaxation	collision.h	608–614
3.92	AO: $\omega_{C\geq 2} = 1$, $A = B = 0$	collision.h	(compile flag)
3.93–3.102	AO simplified collision	collision.h	500–605
3.103–3.105	WP: $\omega_3, \omega_4, \omega_5(\omega_1, \omega_2)$	collision.h	91–119
3.106–3.107	WP: A, B coefficients	collision.h	140–165
3.108–3.110	High-Re limit ($\omega_{3,4,5} \to 0$)	(theoretical)	—
3.111–3.114	Lambda limiter $\tilde{\omega}$	collision.h	172–178, 416–431
3.115	$\lambda = 10^{-2}$ default	collision.h	46–48
3.116–3.118	Simplified diag 4th (ω_6=ω_7=1)	special case of	L524–564
3.119–3.121	Simplified off-diag 4th (ω_8=1)	special case of	L569–584
J.16	Cum.→CM off-diag 4th (forward)	collision.h	267–272
J.16 inv	CM→Cum off-diag 4th (inverse, **with products**)	collision.h	622–637
J.17–J.19	Higher-order cum.↔CM	collision.h	275–321, 641–685
§3.2.1	Strang-splitting sign flip	collision.h	699–701

10 Errata and Bug Fixes

Errata 1: Thesis Eq. 3.89 — $C_{110} \leftrightarrow C_{011}$ Swapped

The thesis prints:

$$\partial_y w + \partial_z v = -3\omega_1/\rho \cdot C_{110}\,, \quad \partial_x v + \partial_y u = -3\omega_1/\rho \cdot C_{011}\,.$$

The correct Chapman–Enskog mapping is:

$$\begin{aligned} \partial_y w + \partial_z v &= -3\omega_1/\rho \cdot C_{011} \quad (\text{yz shear} \to C_{011})\,, \\ \partial_x v + \partial_y u &= -3\omega_1/\rho \cdot C_{110} \quad (\text{xy shear} \to C_{110})\,. \end{aligned}$$

The code uses the corrected form. Verified against walberla's CellwiseSweep implementation.

Fix 1: WP Off-Diagonal Back-Conversion Missing Product Terms

The WP path originally assigned cumulant values directly to central moment slots: `m[I_cbb] = wp_C211_star` (missing the Eq. J.16 inverse products). This caused $O(\mathrm{Ma}^2)$ errors. Fixed by adding the full cumulant→central moment product terms:

```
m[I_cbb] = wp_C211_star + ((m[I_caa]+1.0/3.0)*m[I_abb]
           + 2.0*m[I_bba]*m[I_bab]) * inv_rho;
```

Fix 2: Generalized ω_6, ω_7, ω_8

Previously, $\omega_6 = \omega_7 = \omega_8 = 1$ were hardcoded as `const double omega6 = 1.0`. The 4th-order diagonal used the [GR22] product equilibrium form, which is only valid at $\omega_6 = \omega_7 = 1$. Fixed by:

1. Making $\omega_6/\omega_7/\omega_8$ compile-time configurable via `CUM_OMEGA6/7/8` macros
2. Replacing [GR22] product form with thesis Eq. 3.83–3.85 velocity-derivative form
3. Retaining ω_8 explicitly in Eq. 3.86–3.88

11 Appendix: Index Alias Table

Alias	Array Index	Central Moment $\kappa_{\alpha\beta\gamma}$
`I_aaa`	26	κ_{000} (density fluctuation $\delta\rho$)
`I_baa`	18	κ_{100} (x-momentum)
`I_aba`	14	κ_{010} (y-momentum)
`I_aab`	10	κ_{001} (z-momentum)
`I_caa`	25	κ_{200} (xx-stress)
`I_aca`	24	κ_{020} (yy-stress)
`I_aac`	22	κ_{002} (zz-stress)
`I_bba`	6	κ_{110} (xy-shear)
`I_bab`	4	κ_{101} (xz-shear)
`I_abb`	2	κ_{011} (yz-shear)
`I_bbb`	0	κ_{111}
`I_cba`	13	κ_{210}
`I_bca`	17	κ_{120}
`I_cab`	9	κ_{201}
`I_acb`	8	κ_{021}
`I_bac`	16	κ_{102}
`I_abc`	12	κ_{012}
`I_cbb`	1	κ_{211}
`I_bcb`	3	κ_{121}
`I_bbc`	5	κ_{112}
`I_cca`	23	κ_{220}
`I_cac`	21	κ_{202}
`I_acc`	20	κ_{022}
`I_ccb`	7	κ_{221}
`I_bcc`	15	κ_{122}
`I_cbc`	11	κ_{212}
`I_ccc`	19	κ_{222}